

Reproducible Research

Quarto & Jupyter Notebooks

USP 410/510 | Spring 2026

Dr. Liming Wang

Portland State University

Part 1: From Analysis to Evidence

Based on Turrell, Quarto for Python, and Quarto Jupyter documentation

Today

- What reproducible research means in this class
- How notebooks can help and how they can mislead
- Quarto basics: markdown, code cells, execution, rendering
- Turning exploratory analysis into a report
- Assignment 2: interactive crash map
- Project proposal check-in
- Live demo: notebook/report workflow

Core Claim

A data science product should connect four things:

1. **Question**
2. **Data**
3. **Code**
4. **Claim**

Reproducible work keeps that chain visible.

If the reader cannot trace the claim back to the code and data, the result is weaker than it looks.

Why This Matters for Urban Data Science

Urban data products often inform public decisions.

- Which corridors need safety investments?
- Which neighborhoods face unequal exposure?
- Which transit stops should be prioritized?
- Which policy change appears to have worked?

The stakes make "I ran some code and got this chart" too thin. The work needs a trail.

Reproducibility Is a Spectrum

LEVEL	MEANING	CLASS EXAMPLE
Repeatable	You can rerun it on your laptop	Your notebook runs top-to-bottom
Reproducible	Someone else can rerun it	README + environment + data instructions
Robust	Result survives reasonable choices	Same pattern under alternative filters
Auditable	Reader can inspect assumptions	Narrative explains decisions and limits

For this class, aim for reproducible and auditable.

What Counts as Reproducible Here?

Before submitting an assignment or project milestone:

- Raw data source is named and linked
- Cleaning steps are in code, not manual edits
- Notebook or Quarto document runs from top to bottom
- Figures and tables are generated by code in the document or project
- Package requirements are documented
- AI use is disclosed and checked

Computational Narrative

A notebook or Quarto report is not just code with comments.

It should read like:

1. What question are we asking?
2. What data are we using?
3. What transformations did we make?
4. What did we find?
5. How confident should we be?
6. What should happen next?

Markdown cells carry the argument. Code cells carry the evidence.

The Bad Version

```
df = pd.read_csv("data.csv")  
df = df[df.x > 0]  
df.groupby("area").mean().plot()
```

What is missing?

- What is `x` ?
- Why drop values below zero?
- What does `area` represent?
- Is the mean the right summary?
- What does the chart imply?

The Better Version

```
## Are injury crashes concentrated on high-speed corridors?
```

The ODOT crash file includes a posted speed limit field. I exclude records with missing or zero speed limits because those values do not represent an observed corridor speed.

```
valid_speed = crashes["POST_SPEED_LMT_VAL"].between(5, 80)
speed_summary = (
    crashes.loc[valid_speed]
    .groupby("POST_SPEED_LMT_VAL")["TOT_INJ_CNT"]
    .mean()
)
```

A Simple Report Shape

For most class projects:

SECTION	PURPOSE
Question	State the urban problem clearly
Data	Source, coverage, unit of observation
Methods	Cleaning, filters, transformations
Findings	Figures/tables plus interpretation
Limitations	What the data cannot answer
Next steps	What analysis or action follows

This structure works for notebooks, Quarto reports, and dashboards.

Part 2: Jupyter Notebooks

What Jupyter Is Good For

Jupyter notebooks are excellent for:

- Interactive data exploration
- Teaching and showing code incrementally
- Mixing text, code, output, and charts
- Trying multiple approaches quickly
- Sharing a compact analysis artifact

Notebooks are a workspace and a communication format. That dual role is useful, but it creates risks.

Notebook Failure Modes

- Cells run out of order
- Hidden state from deleted or old cells
- Long notebooks with no section structure
- Large outputs committed into version control
- Important cleaning logic buried in one cell
- "Works after I click around" but not from a clean restart

Rule: if restart-and-run-all fails, the notebook is not done.

Execution Order Matters

This notebook can produce a result that cannot be reproduced:

```
# Cell 5
crashes = crashes[crashes["CNTY_NM"] == "MULTNOMAH"]
```

```
# Cell 2, run later
crashes = crashes.sample(1000)
```

```
# Cell 9
crashes.groupby("CRASH_SVRTY_LONG_DESC").size()
```

The visible order and the executed order are not always the same.

Notebook Hygiene Checklist

Use this before submitting:

- Restart kernel
- Run all cells from top to bottom
- Remove dead experiments or label them clearly
- Put imports and settings at the top
- Use section headings
- Save important outputs to `outputs/`
- Keep reusable functions in `.py` files when they grow

Suggested Notebook Skeleton

```
# Crash Severity Exploration
```

```
## Setup
```

```
Imports, paths, display options
```

```
## Load Data
```

```
Source and file path
```

```
## Clean Data
```

```
Type fixes, filters, missing values
```

```
## Explore
```

```
Tables and plots
```

```
## Findings
```

```
Short written interpretation
```

Naming Helps

Bad names make notebooks harder to audit.

```
df2 = df[df["x"] > 0]
tmp = df2.groupby("y").size()
tmp.plot()
```

Better:

```
valid_speed_crashes = crashes[
    crashes["POST_SPEED_LMT_VAL"].between(5, 80)
]

crashes_by_speed = valid_speed_crashes.groupby(
    "POST_SPEED_LMT_VAL"
).size()
```

Move Stable Logic Out

When a cleaning cell gets long, move it into a function.

```
# crash_cleaning.py
def clean_crashes(crashes):
    out = crashes.copy()
    out["CRASH_DT"] = pd.to_datetime(out["CRASH_DT"], errors="coerce")
    out = out.dropna(subset=["LAT", "LON"])
    return out
```

```
from crash_cleaning import clean_crashes
```

```
crashes = clean_crashes(raw_crashes)
```

The notebook becomes easier to read; the function becomes easier to test.

Exploration vs Submission

EXPLORATION NOTEBOOK	SUBMISSION NOTEBOOK/REPORT
Messy is acceptable	Must run cleanly
Many rough plots	Fewer selected figures
Notes to yourself	Narrative for a reader
Tries alternatives	Explains chosen path
Temporary outputs	Stable outputs

You do not need to preserve every dead end in the final document.

Part 3: Quarto

What Quarto Adds

Quarto turns code plus markdown into polished documents.

- Source file: `.qmd` or `.ipynb`
- Outputs: HTML, PDF, Word, RevealJS slides, websites
- Runs Python, R, Julia, Observable
- Supports figures, tables, citations, parameters, themes
- Works well with Jupyter kernels

For this class: Quarto is a strong default for final reports.

Quarto File Anatomy

```
title: "Crash Patterns Near Schools"
author: "Your Name"
format: html
jupyter: python3
```

```
## Question
```

```
Which school areas have the highest concentration of severe crashes?
```

```
import pandas as pd
import geopandas as gpd
```

YAML controls document settings. Markdown controls narrative. Code controls computation.

Render Command

```
quarto render crash_report.qmd
```

Useful variants:

```
quarto render crash_report.qmd --to html  
quarto render crash_report.qmd --to pdf  
quarto preview crash_report.qmd
```

HTML is the easiest default. PDF may require a LaTeX installation.

Code Cell Options

Put options at the top of a Quarto code cell:

```
#| label: fig-hourly-crashes  
#| fig-cap: "Crash count by hour of day, Oregon 2023"  
#| echo: false  
  
sns.countplot(crashes, x="CRASH_HR")
```

OPTION	MEANING
<code>echo: false</code>	Hide code, show output
<code>warning: false</code>	Hide warnings
<code>message: false</code>	Hide package messages
<code>fig-cap</code>	Add a figure caption
<code>label</code>	Stable reference name

When to Show Code

Show code when:

- The audience needs to inspect methods
- The code is short and clarifies the result
- The assignment asks for code

Hide code when:

- The output is the focus
- The code is long and distracts from the finding
- You are producing a public-facing report

For this class, include enough code that I can verify your workflow.

Figures in Quarto

```
import matplotlib.pyplot as plt
import seaborn as sns

fig, ax = plt.subplots(figsize=(8, 4))
sns.countplot(crashes, x="CRASH_HR", ax=ax)
ax.set_xlabel("Hour of day")
ax.set_ylabel("Crashes")
ax.set_title("Oregon crashes by hour, 2023")
fig.tight_layout()
```

Polish figures in code. Do not edit exported images manually.

Tables in Quarto

```
summary = (  
  crashes  
  .groupby("CRASH_SVRTY_LONG_DESC")  
  .agg(  
    crashes=("CRASH_ID", "count"),  
    mean_injuries=("TOT_INJ_CNT", "mean"),  
  )  
  .sort_values("crashes", ascending=False)  
)  
  
summary
```

Small tables belong in reports. Large tables usually belong in downloadable files.

Quarto From Jupyter

You can render an existing notebook:

```
quarto render crash_analysis.ipynb --to html
```

Or convert later:

```
quarto convert crash_analysis.ipynb
```

Practical workflow:

1. Explore in `.ipynb`
2. Clean up the story
3. Render with Quarto
4. Submit the rendered output and source file

Citations and Links

At minimum, link your data sources in the text.

```
Crash records come from the  
[Oregon Department of Transportation crash data products](https://www.oregon.gov/odot/Data/Pages/CrashDataProducts.aspx)
```

For formal papers, Quarto can use BibTeX:

```
bibliography: references.bib
```

```
Prior work links nighttime conditions to pedestrian risk [@nyt_pedestrians_2023].
```

Parameters

Parameters let you rerun one report for different places or years.

```
title: "Crash Summary"  
params:  
  county: "MULTNOMAH"  
  year: 2023
```

```
county_crashes = crashes[  
  (crashes["CNTY_NM"] == params["county"]) &  
  (crashes["CRASH_DT"].dt.year == params["year"])  
]
```

This is useful for reusable city or corridor reports.

Quarto vs Jupyter

USE CASE	BETTER DEFAULT
Fast exploration	Jupyter
Teaching live code	Jupyter
Final report	Quarto
Public HTML report	Quarto
Interactive dashboard	Streamlit or Quarto dashboard
Static PDF handout	Quarto

The tools overlap. Choose based on the audience and final artifact.

Part 4: Assignment 2 and Projects

Assignment 2: Interactive Crash Map

Assigned today: Thursday, May 7, 2026

Due: Thursday, May 21, 2026

Task: create an interactive map that goes beyond data browsing.

- Pick a specific question, audience, or decision context
- Use ODOT crash data
- Build with `folium`, `plotly`, `streamlit`, or a similar tool
- Export or deploy the result as HTML or a web app

A2 Needs a Point of View

Weak:

Here is every crash point in Oregon.

Stronger:

Where are the most dangerous pedestrian corridors in East Portland?

Stronger:

Which school areas have the highest severe-crash risk within a 10-minute walk?

The map should answer a question, not just display points.

A2 Workflow

1. Choose user/audience
2. Choose geography and time range
3. Clean and filter crash records
4. Add spatial context if needed
5. Design map layers and popups
6. Export or deploy
7. Write short explanation and AI use statement

Keep raw data, processed data, code, and output separate.

Map Tool Options

TOOL	STRENGTH	OUTPUT
<code>folium</code>	Leaflet maps from Python	Standalone HTML
<code>plotly</code>	Interactive plots and map points	HTML / dashboard
<code>geopandas</code>	Spatial cleaning and joins	Data prep
<code>streamlit</code>	Simple web app controls	Hosted app / local app

Week 7 will cover spatial data in more detail. For A2, start with a focused map question now.

Minimal Folium Export

```
import folium

m = folium.Map(location=[45.52, -122.67], zoom_start=11)

for _, row in crashes.dropna(subset=["LAT", "LON"]).iterrows():
    folium.CircleMarker(
        location=[row["LAT"], row["LON"]],
        radius=3,
        popup=row["CRASH_SVRTY_LONG_DESC"],
        fill=True,
    ).add_to(m)

m.save("outputs/crash_map.html")
```

Minimal Plotly Export

```
import plotly.express as px

fig = px.scatter_map(
    crashes,
    lat="LAT",
    lon="LON",
    color="CRASH_SVRTY_LONG_DESC",
    hover_data=["CRASH_DT", "POST_SPEED_LMT_VAL"],
    zoom=10,
    height=650,
)

fig.write_html("outputs/crash_map.html")
```

This creates a shareable HTML file.

Project Proposal Due Today

Your proposal should answer:

- What urban question are you studying?
- Why does it matter?
- What data will you use?
- What methods or visualizations are likely?
- What is your intended final product?
- What risks or unknowns remain?

Keep the scope small enough to finish by Week 11.

Proposal Test

Can your project question produce a specific deliverable?

TOO BROAD	MORE WORKABLE
Housing affordability in Portland	Where did rent burden rise most from 2018-2023?
Transit access and equity	Which low-income areas have weak access to frequent service?
Traffic safety	Which corridors have high severe-crash density after dark?
Climate and cities	Which blocks have high heat exposure and low tree canopy?

Specific questions make better data work.

AI Use Statement

For assignments and projects, include a short statement:

```
## AI Use Statement
```

```
I used [tool name] to help with [tasks]. I checked the outputs by  
[verification steps]. I am responsible for the final code, analysis,  
and interpretation.
```

AI assistance is allowed and encouraged. Unverified AI output is not enough.

Part 5: Live Demo

Demo Goal

Turn exploratory work into a reproducible report:

1. Start from an existing crash analysis notebook
2. Add section headings and narrative
3. Clean execution order
4. Add figure captions
5. Render to HTML with Quarto
6. Check the output as a reader

Demo Commands

```
quarto preview crash_report.qmd  
quarto render crash_report.qmd --to html
```

If starting from a notebook:

```
quarto render crash_analysis.ipynb --to html
```

After rendering, open the HTML and read it without looking at the source.

What We Check

- Does the document answer the stated question?
- Can a reader tell where the data came from?
- Are cleaning choices explained?
- Do figures have titles, labels, and captions?
- Are warnings/errors absent or intentional?
- Does the document run from a clean session?

In-Class Exercise (10-15 min)

Take one analysis from Assignment 1 or your project idea.

1. Write one clear research question
2. Add a short data source paragraph
3. Create one table or figure
4. Write two sentences interpreting it
5. Restart and run all cells

Optional: render it with Quarto.

Submission Checklist

For any notebook or Quarto submission:

- Source file included (`.ipynb` or `.qmd`)
- Rendered output included (`.html` , `.pdf` , or `.docx`) when requested
- Code runs from top to bottom
- Paths are relative to the project folder
- Data source is documented
- AI use statement is included
- Final claims match the figures/tables

Key Takeaways

- Reproducibility is about traceability from claim to code to data
- Notebooks are powerful but need clean execution discipline
- Quarto is a strong tool for turning analysis into reports
- Assignment 2 needs an interactive map with a clear question
- Your project proposal should be specific enough to build

For Next Week

Week 7 topic (Thursday, May 14, 2026): Working with spatial data and maps

Read:

- Rey, Arribas-Bel, and Wolf, Geographic Data Science with Python, Chapters 1-4

Due:

- DataCamp 3: Intro to Data Visualization with Seaborn